

A Self-Contained Cubical Type Theory Kernel in Lean 4: Results, Mathematical Significance, and Relevance to Open Problems in Logic and Higher Category Theory

The FormalizedMathematics Project

Last updated: 2026-07-11 (living document, updated as the project advances)

Abstract

This project implements, from scratch and with no external mathematical libraries, a proof assistant kernel for a CCHM-style cubical type theory inside Lean 4, together with a library of roughly 250 machine-checked theorems of synthetic homotopy theory, univalent mathematics, and category theory carried out *inside* the object language. Highlights include a univalence axiom that *computes*, the complete quantified theorem $\pi_1(S^1) \cong \mathbb{Z}$, the identification $\Omega S^1 \equiv \mathbb{Z}$ as an equality of types, $T^2 \simeq S^1 \times S^1$ with every cell definitional, the general graduate lemma (`isoToEquiv`), six higher inductive types including Eilenberg–MacLane spaces $K(G, 1)$, Mac Lane’s pentagon and triangle coherence in 1-types, and the 2-categorical structure of the category of categories. This document records the deliverables, explains their mathematical significance, and gives an honest assessment of how the platform bears on open problems: the homotopy hypothesis, the axiomatization of coherence, and the comparison problem for weak higher categories.

Contents

1	Overview	2
2	The kernel: features and design principles	2
2.1	Core type theory	2
2.2	Higher inductive types	3
2.3	Design principles discovered	3
3	The library: principal results	3
3.1	Path algebra and univalence	3
3.2	h-levels	4
3.3	The circle: a complete computing π_1	4
3.4	Integers, arithmetic, quotients	4
3.5	Category theory in a univalent universe	4
3.6	Higher inductive theorems	4
3.7	Coherence, chapter 1	5
3.8	Coherence, chapters 2–4: untruncated tools and Eckmann–Hilton	5
4	Engineering results of independent interest	7
5	Mathematical significance	7

6	Relevance to open problems	8
6.1	The homotopy hypothesis	8
6.2	Axiomatization of coherence	9
6.3	Weak higher categories and the comparison problem	9
6.4	What would constitute progress here	9
7	Current frontier	9

1 Overview

The project consists of two strictly separated layers.

The kernel (metatheory, in Lean 4). A bidirectional type checker, a normalization-by-evaluation (NbE) evaluator, and a de Morgan interval algebra implementing the Cohen–Coquand–Huber–Mörtberg (CCHM) cubical type theory: dependent products and sums, a cumulative universe hierarchy $\mathcal{U}_0 : \mathcal{U}_1 : \dots$, path types **PathP**, Kan operations (**transp** with a constancy rule, **hcomp** with face systems), Glue types, and a growing family of (higher) inductive types. The kernel is $\sim 5,000$ lines of Lean with *no* dependency on mathlib; every judgment is decided by executable code.

The library (object language). Roughly 250 definitions and theorems, each a closed term of the object type theory together with its type, verified by the kernel at build time. Nothing in the library is a Lean theorem *about* the kernel; it is mathematics carried out *inside* the implemented type theory, in the same sense that the Cubical Agda library is mathematics inside Cubical Agda.

This two-level architecture — a strict metatheory (Lean) hosting a univalent object theory (cubical) — is itself of foundational interest; see §??.

2 The kernel: features and design principles

2.1 Core type theory

- De Morgan interval: free distributive lattice with involution on interval variables; equality decided by canonical antichain disjunctive-normal forms.
- NbE with defunctionalized closures: values are first-order data; ~ 90 special closure constructors encode the Kan structural rules (transport in $\Pi/\Sigma/\mathbf{PathP}$ families, tube transformers for eliminator/composition commutation, motive codomains of dependent eliminators).
- Bidirectional checking; path application **papp** carries endpoint annotations, which is what makes neutral boundary computation sound under untyped evaluation.
- Universes with judgmental cumulativity; sort inference.
- Kan operations: **transp** with the *constancy rule* (transport along a constant family is the identity, decided by an occurs-check on the semantic family), **hcomp** over face systems given by conjunctions of ($i=0/1$) constraints, derived **hfill** and heterogeneous composition.

- Glue types with `unglue`; univalence as the *program* `ua`; composition-of-universe (`hcomp` at $\mathcal{U}$) computed as a Glue along branch lines glued by a checker-verified `lineEquiv`; the general CCHM §6.2 `transpGlue` including fiber corrections.

2.2 Higher inductive types

Eight (higher) inductive types are implemented, each contributing a new implementation pattern:

S^1	first HIT; lazy endpoint collapse; eliminator/hcomp commutation
ΣA (suspension)	first <i>parameterized</i> HIT; structural transport
T^2 (torus)	first <i>2-cell</i> constructor
$\ A\ $ (prop. truncation)	first <i>recursive</i> path constructor
pushouts	cells carrying function <i>annotations</i> (endpoints not recoverable)
A/R (set quotients)	<i>dependent</i> eliminator; 2-dimensional truncation cell
$K(G, 1)$ (Eilenberg–MacLane)	composition 2-cell, 3-dimensional truncation cell
lists, sums, $\top$, $\perp$, $\mathbb{N}$, $\mathbb{Z}$	ordinary inductives (in NbE style)

2.3 Design principles discovered

Three principles were forced on us by bugs and are now systematic:

1. **Lazy boundary collapse.** Eagerly reducing `loop 0` $\mapsto$ `base` (and its relatives) destroys the Kan structure that `transpGlue` later needs; all endpoint collapses happen in a single `force` function, and eliminators match *unforced* cell values first. Five independent bugs traced to violations of this principle.
2. **Annotation-carrying cells.** When a cell’s boundary mentions parameters not recoverable from the cell (pushout’s `push c` has endpoints `inl(f c)`), the constructor value carries the needed data, and the checker verifies consistency with the ambient type.
3. **Off-face junk tolerance.** Partial elements are computed unrestrictedly and are meaningless off their faces; every eliminator now passes such junk through instead of panicking, making the evaluator total on the values that occur-check walks can reach.

3 The library: principal results

3.1 Path algebra and univalence

Function extensionality, path induction J (*derived* as a transport along a connection square, not primitive), the full groupoid structure of paths (units, associativity, inverses — the inverse laws by connection-shaped fillers), two-dimensional path algebra, and the computation rules one hopes for:

$$\text{trans refl refl} \longrightarrow \text{refl} \quad (\text{at concrete types}), \quad \text{transport (ua } e) \longrightarrow e.\text{fst}.$$

On top of `ua`: `ualdEquiv : ua(idEquiv)  $\equiv$  refl` by a single Glue square, and the η -rule `uaEta : ua(pathToEquiv p)  $\equiv$  p` by J one universe up — one half of the full univalence equivalence, from which `isSetPathU` (paths between sets in $\mathcal{U}$ form a set) follows by a retract argument.

3.2 h-levels

Contractibility, propositions, sets, groupoids; singletons are contractible; `isPropsContr` (the classical four-tube `hcomp`); being an equivalence is a proposition; being a set is a proposition; closure of h-levels under Π , Σ (with propositional fibers), retracts; **Hedberg’s theorem** (decidable equality implies sets) — whose proof exercises the derived J , the inverse laws, and case analysis with the “inspect” idiom; `setIsoToEquiv` (isomorphisms into sets are equivalences, by a square-filling in the set); and the **general graduate lemma** `isoToEquiv` with no h-level hypothesis, implementing the classical `lemIso`: three `hfills` and two correction squares, with compound faces $(k \wedge j)=1$ encoded as conjunctions and $(k \wedge j)=0$ as split branches.

3.3 The circle: a complete computing π_1

- The full quantified theorem $\pi_1(S^1) \cong \mathbb{Z}$: both round trips, `encode/decode` with `decode` a genuine dependent function over S^1 built from a Π -level composition (working around a documented checker incompleteness) and cancellation-path corrections.
- *Computation*: `winding(loop · loop) → +2`, `winding(loop-1) → -1`, etc., running through `HCompU` and `Glue transport` — among the few implementations in existence where these classical numbers are *computed* by univalence rather than proved.
- $\Omega S^1 \equiv \mathbb{Z}$ as an identification of types in the universe (`ua` of the loop-space equivalence).
- Negative results: `loop ≠ refl`, hence S^1 is not a set; `isSet( $\Omega S^1$ )` by a retract of $\mathbb{Z}$; S^1 is connected ($\forall x. \| \text{base} \equiv x \|$).

3.4 Integers, arithmetic, quotients

$\mathbb{Z}$ with the substitution-stable `pos/negsuc` representation (the strict-cancellation representation was proved *substitution-unstable* — a rediscovery, by mechanized failure, of why Cubical Agda avoids it); commutative group laws all by induction; decidable equality for $\mathbb{N}$, $\mathbb{Z}$, booleans, letters, lists (with Hedberg giving second, independent `isSet` proofs); the classical difference construction $(\mathbb{N} \times \mathbb{N})/\sim \simeq \mathbb{Z}$ exercising the full quotient workflow (well-definedness, `elimProp`, equivalence).

3.5 Category theory in a univalent universe

Monoids and the monoid $(\mathbb{Z}, +, 0)$; universe-indexed precategories (objects in $\mathcal{U}_{n+1}$, homs in $\mathcal{U}_n$); the category of types (laws definitional by η); the fundamental groupoid of a type; functors and natural transformations; **functor categories** — necessarily one universe level up, with naturality transported as a *proposition* over hom-sets; vertical and horizontal composition, whiskering, and the **interchange law**: the 2-categorical structure of **Cat** machine-verified. Concretely: the delooping $B\mathbb{Z}$ (one object, homs $\mathbb{Z}$, composition $+$) and the endofunctor category $[B\mathbb{Z}, B\mathbb{Z}]$.

3.6 Higher inductive theorems

- $S^0 \simeq \text{Bool}$ and $S^0 \equiv \text{Bool}$, with `transport` computing `north ↦ true`; the famous demonstration `transport(ua not) true → false`.
- $S^2 := \Sigma S^1$; suspension functorial; the map $\sigma : S^1 \rightarrow \Omega S^2$ underlying the generator of $\pi_2(S^2)$.

- $T^2 \cong S^1 \times S^1$ (and, via the general graduate lemma and `ua`, $T^2 \simeq S^1 \times S^1$ and $T^2 \equiv S^1 \times S^1$): the celebrated cubical proof where the square cell corresponds to the pair of loops and *every cell of both round trips is definitional*.
- $\Sigma A \simeq$ pushout of $\top \leftarrow A \rightarrow \top$ (again all-definitional); wedges $A \vee B$ and mapping cones as one-liners; the figure eight $S^1 \vee S^1$ with a rank-2 cover computing the *abelianized* winding $\pi_1(S^1 \vee S^1) \rightarrow \mathbb{Z}^2$ — with the computed fact that this invariant cannot distinguish LR from RL , a machine demonstration that it only sees the abelianization of F_2 .
- Propositional truncation: `isProp` $\|A\|$, functoriality, idempotence $\| \|A\| \| \simeq \|A\|$; *mere existence* statements (S^1 connected, winding merely surjective); $\|A\| \simeq A / \sim_{\text{total}}$, identifying truncation as a quotient.
- Free-group groundwork: reduced words over two generators; the cancellation prepend `consG` proved an equivalence *generically in the generator*, with case rewriting by *connections* ($e@(i \wedge j)$) instead of J ; the rank-2 free cover `helixF2` and the non-abelian winding `windF2` (generator computations verified; deep composites pending performance work).
- Eilenberg–MacLane spaces: the $K(G, 1)$ HIT with composition 2-cell and truncation 3-cell; `emloop` is a group homomorphism (a `compPath`-uniqueness cube against the `emcomp` cell); `emloop1` $\equiv$ `refl` from the group laws and the groupoid laws of paths; the `Codes` family over $K(G, 1)$ into the *groupoid of sets* (with `isGroupoidHSet` proved via the carrier-path retraction and `isSetPathU`); `uaCompMul` — `ua` of composed right-multiplications — verified (108s in the native runner); `encode/decode`/ $\pi_1(K(G, 1)) \simeq G$ fully constructed, final checks running on the native infrastructure.
- **The two circles are one:** $S^1 \simeq K(\mathbb{Z}, 1)$ fully constructed — `toEM` (`loop` $\mapsto$ `emloop1`), `fromEM` (recursor with the `intLoopComp`-corrected composition cell), and both round trips; the key lemma `tolntLoop` (`cong toEM` distributes over composition *definitionally*, because constant-motive eliminators commute with `hcomp`) verifies in 3 ms natively. The heavy round-trip checks run on the native infrastructure.

3.7 Coherence, chapter 1

Mac Lane’s **pentagon** and **triangle** for path composition, stated with the full associator/whiskering composites and proved *in any 1-type* (parallel 2-cells in a groupoid agree), with the $\mathbb{Z}$ -instance. A finding of independent interest: the *untruncated* pentagon cannot be proven by the naive quadruple path induction, because `transrefl refl` reduces to `refl` only at concrete types; at abstract types the composite is a stuck `hcomp`, so the coherence cells (`assocRefl`, unit-cell collapses, ...) must be bootstrapped by hand — a precise computational location of the difficulty that coherence theory addresses.

3.8 Coherence, chapters 2–4: untruncated tools and Eckmann–Hilton

Beyond the 1-type case, the library now proves coherence cells *in an arbitrary type*, with connections only (no J , no extra Kan compositions):

- **Definitional endpoint fillers.** `transFill` : $\text{PathP}(\lambda k. w \equiv qk) p(p \cdot q)$ and its left counterpart, engineered so that an identically-true tube face makes the endpoints collapse *definitionally*; the untruncated associator `assocConn` : $p \cdot (q \cdot r) \equiv (p \cdot q) \cdot r$ then assembles by composing the two fillers across the *moving middle point* qk , with no cube reasoning escaping the fillers.

- **A definitional unit coincidence.** transReflR refl and transReflL refl are *convertible* (the 3-cell is refl): the two unit squares have all-constant tubes at refl . In book HoTT this coincidence is a nontrivial lemma; here the kernel computes it away.
- **The square-exchange principle.** For any syntactic square Skm , the two boundary routes agree: $(\text{bottom} \cdot \text{right}) \equiv (\text{left} \cdot \text{top})$, proved by sliding the middle point along the anti-diagonal $S(-m)m$ with connection half-squares. Instances: the cong_2 exchange law (congSlide) and the naturality of both unitors.
- **Eckmann–Hilton.** $\alpha \cdot \beta \equiv \beta \cdot \alpha$ for 2-loops in *any* type — the classical first theorem of synthetic homotopy theory beyond $\pi_1(S^1)$ — as an 18-step composite of 3-cells: exchange the two loops with congSlide , restore whiskered forms with the unitor naturalities, and cancel the shared unitor (the definitional unit coincidence keeps the left/right bookkeeping inside conversion). Verified in milliseconds by the type checker. Packaged algebraically: *the second loop space of any type is a commutative monoid* (untruncated), with multiplication composition, commutativity Eckmann–Hilton — instantiated at every rung of the loop-space tower.
- **Canonical face literals and Mac Lane’s triangle.** A kernel improvement (canonical atomic face literals in system normalization: $\neg k=1 \mapsto k=0$, conjunctive splitting) makes the fillers of $\text{assocConn} p \text{refl} q$ *convertible* with the two unitors — the associator at a middle refl is the diagonal of the unitor square. Mac Lane’s **triangle** then holds untruncated, in any type, by a three-step argument (a diagonal-vs-edges lemma by the moving-middle method, plus right cancellation).
- **Mac Lane’s pentagon, untruncated.** The full pentagon

$$\alpha_{p,q,r \cdot s} \cdot \alpha_{p \cdot q,r,s} \equiv (p \cdot \alpha_{q,r,s}) \cdot \alpha_{p,q \cdot r,s} \cdot (\alpha_{p,q,r} \cdot s)$$

holds in *any* type, with $\alpha := \text{assocConn}$. Proof architecture: three naturality squares obtained by *moving the associator along its own fillers* (each a one-line application of the square-exchange principle), a *fusion cell* built with conjunction faces — a face $(l=0) \wedge (m=1)$ is identically false on one side of the interpolation and normalizes to $(l=0)$ on the other, so cells whose face systems differ across the two ends become constructible — and a final *box-with-lid* cell in which all four edges of the connecting cube are collapsed by identically-true faces whose walls are the fillers themselves. Supporting kernel work: the checker’s face resolution was generalized to arbitrary interval expressions via canonical-DNF disjunct covers. Together with the triangle this completes **Mac Lane’s coherence pair, untruncated, axiom-free**.

- **Dimension-1 homotopy hypothesis: the counit.** For any pointed 1-type (A, a) the loop space $\Omega(A, a)$ carries a group structure in which *every law is one of the untruncated coherence cells above* (associativity is assocConn , the units are the two unitors, the inverse laws are cancell); the 1-type hypothesis enters only through set-ness. The realization map $K(\Omega(A, a), 1) \rightarrow A$ is ‘ emlrec ’ with the *identity* loop map, whose coherence cell is exactly the composition filler transFill — and on loops it computes to the identity *definitionally* (refl proof). This generalizes $S^1 \simeq K(\mathbb{Z}, 1)$ and opens the programme $A \simeq K(\pi_1 A, 1)$ for pointed connected 1-types.
- **The fundamental groupoid, functorially.** Internal (multi-object) groupoid structure is defined; every 1-type carries it on its path spaces with *every* law an instance of the untruncated coherence cells, the 1-type hypothesis entering only through set-ness of the homs. The theory

is developed functorially: groupoid morphisms, identity and composition of morphisms, and the functorial action of any map ($F_1 = \text{cong}$, identity preservation definitional, composition preservation by congTrans). Two strictness theorems hold *by* ‘ refl ’: the vertex group of the fundamental groupoid *is* the loop group, and $\text{cong}(g \circ f) \equiv \text{cong}g \circ \text{cong}f$. With groups as one-object groupoids this completes the 1-type-to-groupoid direction of the “groupoids $\simeq$ 1-types” roadmap item; the reverse direction’s prerequisite — a *general-groupoid Eilenberg–MacLane HIT $B\mathcal{G}$* — is now implemented in the kernel: formation, point/arrow/composition/ truncation constructors, and a recursor that computes on points, with smoke tests over $B(\text{Unit}, \mathbb{Z}, +)$ all kernel-checked and the entire extension validated behaviour-preserving by the differential harness. On this foundation the **dimension-1 classification theorem** is now *fully machine-verified, with no pointedness or connectedness hypotheses*:

$$B(\Pi_1 A) \simeq A \quad (\text{hence } B(\Pi_1 A) \equiv A \text{ by univalence})$$

for *every* 1-type A . The proof is light precisely because the HIT carries the right universal property: the counit is the recursor with the composition filler as its coherence cell and computes definitionally on points and arrows; the unit is the dependent eliminator with ‘ $\text{barr } \text{cong } \text{bpt}$ ’ (proved by cancellation and J) as its arrow cell — no encode–decode, no transport along ‘ ua ’ lines.

4 Engineering results of independent interest

- **Native versus interpreted checking.** Build-time guards run in Lean’s elaborator interpreter; moving deep checks to a compiled test runner yields $\sim 100\times$: the check of `uaCompMul` went from > 6.75 hours (unfinished) to 108 seconds. `implemented_by` native overrides are silently inert in the elaborator — a fact that had invalidated several earlier optimization experiments and is worth knowing for any Lean-hosted evaluator.
- **Definition environments.** References to already-verified definitions are *trusted* (checked once, in dependency order), the standard architecture of proof assistants, implemented as a one-node extension; and an alternative environment-injection mode in which references resolve to de Bruijn variables over a preloaded, value-shared context.
- **A catalogue of failure modes** for evaluator caching (GC address reuse causing unsound hits; capture-based over-approximation in occurs checks breaking *load-bearing* definitional equalities), each documented with its mechanism.

5 Mathematical significance

1. **Univalence computes.** The kernel is a working demonstration — independent of Cubical Agda and `cubicaltt` — that the univalence axiom can be a program: transport along `ua(e)` *evaluates* to e , winding numbers are computed, and the classical `transport(ua not)` example runs. This is evidence in the tradition of constructive canonicity for cubical type theories, obtained by implementation.
2. **Synthetic homotopy theory, effectively.** The $\pi_1(S^1)$ development realizes the Licata–Shulman encode–decode method in a new implementation, and pushes further to identifications in the universe ($\Omega S^1 \equiv \mathbb{Z}$, $T^2 \equiv S^1 \times S^1$). The figure-eight computations exhibit *effective* covering-space theory: invariants evaluated, not just constructed.

3. **The classifying-space programme.** $K(G, 1)$ with its `Codes` family is the level-1 core of “groups are pointed connected 1-types”. Everything needed — `isGroupoidHSet`, univalence η , the h-level tower — was built from first principles inside the object theory.
4. **Coherence made computational.** The pentagon/triangle development, and especially the *negative* finding about untruncated base cases, gives a sharp computational articulation of what coherence data is: exactly the cells that do not come for free from Kan operations at abstract types.
5. **Metatheoretic data points.** The substitution-instability of strictly-cancelling $\mathbb{Z}$, the necessity of lazy boundary collapse, and the junk-tolerance requirement are small theorems-by-implementation about cubical evaluators that complement the paper literature.

6 Relevance to open problems

We state plainly: *this project has not solved, and does not claim to solve, any of the open problems below*. Its value is as a platform on which well-defined fragments can be formulated and machine-checked, and as a source of computational evidence. With that stated:

6.1 The homotopy hypothesis

For Kan complexes the homotopy hypothesis is classical; the open formulations concern Grothendieck ∞ -groupoids (Maltsiniotis, Ara, Henry) and, inside type theory, the impossibility to date of even *stating* the full hypothesis internally, which reduces to the open problem of defining semi-simplicial types in HoTT. The project bears on this in three ways.

1. *The level-1 fragment as theorems.* “Groups $\simeq$ pointed connected 1-types” decomposes into machine-checkable pieces: $\pi_1(S^1) \cong \mathbb{Z}$ (done), $K(G, 1)$ with $\pi_1(K(G, 1)) \simeq G$ (constructed; final checks running), $S^1 \simeq K(\mathbb{Z}, 1)$ (constructed), and — new — **the dimension-1 homotopy hypothesis in full**: for every pointed *connected* 1-type (A, a) ,

$$K(\Omega(A, a), 1) \equiv A,$$

via the realization map (‘`emlrec`’ with the identity loop map, whose coherence cell is the composition filler) shown to be an equivalence: $\Omega(\text{cong})$ agrees with ‘`encode`’ by a two-step chain, is therefore an equivalence on loops, extends to all endpoints by connectedness of $K(G, 1)$ (itself proved with no group laws), and yields contractible fibers through a pair-path whose second component is a corrected singular filler. The whole chain is constructed and compiles; the heavy conversion checks run in the native verifier. Likewise $\pi_1(S^1 \vee S^1) = F_2$ (the free group on two letters, as reduced words) is fully constructed: the covering space, both winding directions — the kernel computes transport along even the conjugated right-circle generators definitionally — and both round trips, the encode direction by word induction through a no-cancellation lemma. “Groupoids $\simeq$ 1-types” remains on the roadmap.

2. *Two-level architecture.* The known obstruction to internalizing ∞ -structures is infinite coherence; the current best strategy is two-level type theory (Annenkov–Capriotti–Kraus–Sattler). This project *is* structurally a 2LTT: Lean is a strict metatheory in which towers indexed by external $\mathbb{N}$ — e.g. a van den Berg–Garner style ω -groupoid structure on each object-language type, as a Lean function $n \mapsto (\text{cell structure as kernel-checked terms})$ — can be defined and

verified. This is a concrete, feasible research direction enabled by the codebase. This direction is now *begun*: the module ‘LibTower’ defines, as a Lean function of external n , the iterated loop space $\Omega^n(A, a)$ together with its composition, untruncated associator, Mac Lane triangle and pentagon, the middle-four interchange, and (from dimension 2) the Eckmann–Hilton braiding — each rung a one-line instantiation of the generic coherence cells, stated uniformly in the ambient type and kernel-checked per level (verified for $n \leq 3$; the check is cheap at every level). Every object-language type thus demonstrably carries the first rungs of a van den Berg–Garner-style weak ω -groupoid structure. The tower is moreover *functorial*: iterated ‘cong’ gives loop maps $\Omega^n f$ at every level with no conjugation (basepoints are preserved definitionally), and the functor law $\Omega^n(g \circ f) \equiv \Omega^n g \circ \Omega^n f$ holds by ‘refl’ at every level.

3. *Computational evidence*. Where the hypothesis has computational content (winding numbers, transport along identifications), the platform executes it.

6.2 Axiomatization of coherence

Classically, coherence is settled for bicategories and tricategories and open (as a comparison/axiomatization problem) in general. The cubical point of view reframes it: Kan operations generate all compositions and their coherences *uniformly*, and the question becomes which coherence cells are definitional and which require construction. The project’s contribution is a precise experimental map of that boundary: groupoid-level coherence is a one-liner (1-types); untruncated coherence requires bootstrapped cells whose exact list (assocRefl, unit collapses at abstract types, ...) is now identified. Extending this map upward (pentagonators, ...) on this platform is mechanical-but-informative work that would feed any future axiomatization: it distinguishes, with machine certainty, *free* coherence from *axiomatic* coherence.

6.3 Weak higher categories and the comparison problem

The library realizes the low-dimensional fragment: (pre)categories, the 2-category **Cat** with interchange, hom-set-truncated functor categories, and the groupoid **hSet**. The comparison problem for definitions of weak n -category is untouched — honestly, it is out of reach of any current proof assistant for general n — but the platform supports two relevant activities: (i) formalizing particular definitions at low n and proving instances of comparisons; (ii) using the meta-level (Lean) to manage the coherence bookkeeping that blocks internal statements, per the 2LTT remark above.

6.4 What would constitute progress here

Concrete, staged milestones this platform can realistically host: $\pi_1(K(G, 1)) \simeq G$ official (running); $S^1 \simeq K(\mathbb{Z}, 1)$; groupoids $\simeq$ 1-types; a meta-level ω -groupoid tower on object types (2LTT-style van den Berg–Garner); the untruncated pentagon via bootstrapped cells; $\pi_1(S^1 \vee S^1) \simeq F_2$; and, as a long-range target, $\pi_2(S^2) \simeq \mathbb{Z}$ (Freudenthal-grade).

7 Current frontier

As of this writing: the $\pi_1(K(G, 1))$ verification chain runs on the native infrastructure (uaCompMul verified; remaining checks in progress); evaluator performance science continues (value sharing; conversion caching); see `HANDOFF.md` in the repository for the day-level state. This document is maintained alongside the code and updated at each substantive milestone.